

Redis Streams CLI 실습

XADD · XREADGROUP · XACK · XPENDING · XAUTOCLAIM

개요 10분 + 실습 50분 · Day 02

PEL 직접 확인

Consumer crash 복구

수평 확장 분배

COINCRAFT
ACADEMY

이 세션이 답하는 질문

THREE KEY QUESTIONS — VERIFIED BY CLI HANDS-ON

Q1. S7에서 배운 PEL이 실제로 생기는가?

CLI에서 XREADGROUP 후 XPENDING을 치면 직접 눈으로 볼 수 있다.

Q2. Consumer가 crash하면 정말로 메시지가 재수신되는가?

XACK 없이 메시지를 읽고 대기하면, XAUTOCLAIM으로 재수신 가능함을 확인한다.

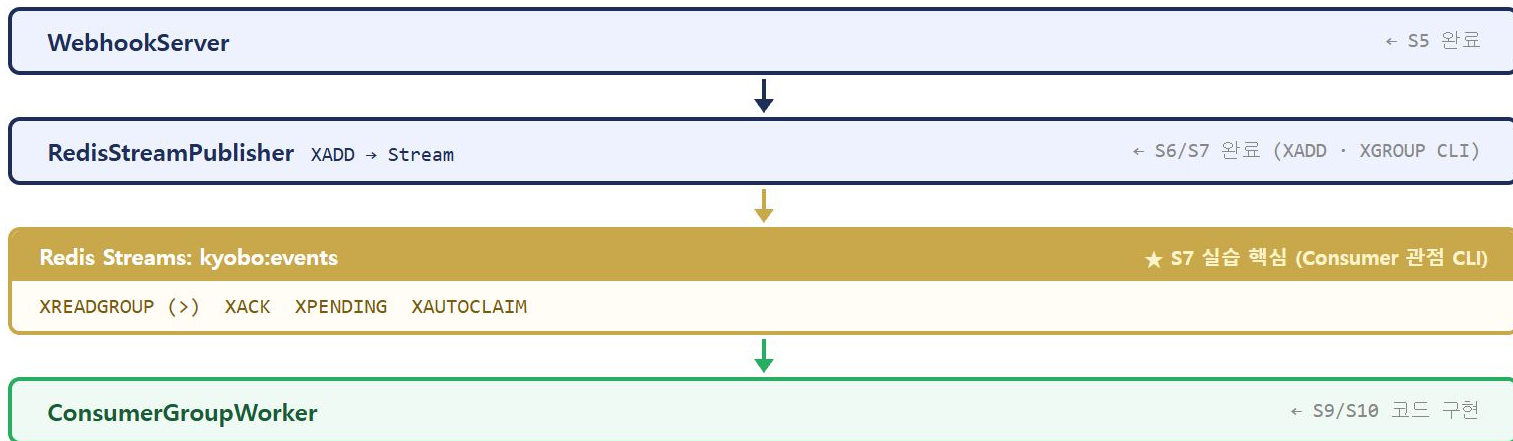
Q3. Consumer 2개가 메시지를 어떻게 나눠 가져가는가?

두 개의 터미널에서 동시에 XREADGROUP을 치면 분배가 보인다.

핵심 원칙 — S7에서 이론으로 배운 PEL·Consumer Group·XAUTOCLAIM을 CLI에서 **눈으로 직접 확인**하는 것이 이 세션의 목적이다.

전체 흐름에서의 위치

THIS SESSION IS THE CONSUMER-SIDE CLI FOR kyobo:events STREAM



이론 → CLI 직접 확인 대응표

WHY HANDS-ON CLI — THEORY BECOMES VISIBLE

S7에서 Redis Streams 내부 구조(PEL, Consumer Group, XAUTOCLAIM)를 이론으로 배웠다. S8의 목표는 그것을 **눈으로 직접 보는 것**이다.

이론에서 배운 것	CLI에서 확인할 것
PEL이 존재한다	XPENDING 결과에서 직접 확인
XREADGROUP 후 PEL 등록	XREADGROUP > 전후로 XPENDING 수 변화 관찰
XACK 후 PEL 제거	XACK 전후 XPENDING 비교
Consumer crash 복구	XAUTOCLAIM으로 소유자 변경 직접 확인
수평 확장 분배	터미널 2개에서 각자 다른 메시지 수신 확인

ConsumerGroupWorker 루프 ↔ CLI 1:1 대응

WORKER INTERNAL LOOP — REPLICATED STEP BY STEP IN CLI

ConsumerGroupWorker.start() 루프

- ① `_reclaimPending()` — `XAUTOCLAIM(idle > minIdleMs)`
- ② `_processNew()` — `XREADGROUP GROUP ... > COUNT n, BLOCK ms`
- ③ `_handleWithRetry(msg)` — `processor.process(msg)` 성공 시 XACK, 실패 시 `retryCount + 1` (XACK 안 함)

실제 `ConsumerGroupWorker`는 `while(running)`으로 자동 반복. CLI로는 루프를 흉내내므로 하나씩 수동 실행.

CLI에서 직접 실행

```
① _reclaimPending() ↔
XAUTOCLAIM kyobo:events
    issuer-consumers consumer-2 0 0-0 COUNT 10

② _processNew() ↔
XREADGROUP GROUP issuer-consumers
    consumer-1 COUNT 10 STREAMS kyobo:events >

③ 성공 → XACK, 실패 → 아무것도 안 함
XACK kyobo:events issuer-consumers <id>
```

CLI ↔ TypeScript 코드 대응표

REDIS CLI COMMANDS MAP TO TYPESCRIPT METHOD CALLS

CLI 명령	TypeScript 메서드	설명
XADD kyobo:events * eventType NFT_ISSUED ...	<code>publisher.publish(event)</code>	이벤트 발행
XGROUP CREATE kyobo:events issuer-consumers 0 MKSTREAM	<code>publisher.initialize()</code>	그룹 생성
XREADGROUP GROUP issuer-consumers consumer-1 COUNT 10 STREAMS kyobo:events >	<code>redis.xreadgroup(group, consumer, ...)</code>	새 메시지 읽기
XACK kyobo:events issuer-consumers <id>	<code>redis.xack(streamKey, groupName, id)</code>	처리 완료 선언
XAUTOCLAIM kyobo:events issuer-consumers consumer-2 0 0-0 COUNT 10	<code>redis.xautoclaim(...)</code>	idle 메시지 인계
XPENDING kyobo:events issuer-consumers - + 10	(모니터링 전용)	PEL 상태 조회

메시지 ID 형식과 실습에서의 활용

MESSAGE ID FORMAT — UNIX TIMESTAMP + SEQUENCE NUMBER

Redis 자동 생성 messageId 구조

1714000001000-0

Unix Timestamp (밀리초)
시스템 시계 기반 자동 생성

Sequence 번호
같은 밀리초 내 순서 보장

CLI 실습에서 ID를 어떻게 쓰는가

```
# XACK: XREADGROUP 반환값을 복붙
XACK kyobo:events issuer-consumers
1714000001000-0

# XAUTOCLAIM: 스트림 처음부터
XAUTOCLAIM kyobo:events issuer-consumers
consumer-2 0 0-0 COUNT 10

# └─ 0-0 = 스트림의 처음
```

실습 주의: 실제 반환되는 ID는 실습 시점 타임스탬프 기반. → XREADGROUP 결과에서 반환된 ID를 직접 복사해서 XACK에 붙여넣어야 함.

실습 전체 구성 — 6 Steps

LAB OVERVIEW — 50 MINUTES HANDS-ON

- Step 1** | Docker로 Redis 기동 (5분)
- Step 2** | XADD로 NFT 이벤트 5개 적재 (10분)
- Step 3** | Consumer Group 생성 + XREADGROUP으로 읽기 (10분)
- Step 4** | XACK + XPENDING으로 PEL 확인 (10분)
- Step 5** | 미ACK 재수신 시뮬레이션 (XAUTOCLAIM) (10분)
- Step 6** | Consumer 2개 동시 실행 → 메시지 분배 확인 (10분 + 5분 정리)

실습 환경 준비 — ConsumerGroupWorker 코드 미리 훑기

PRE-LAB OVERVIEW — 10 MIN · DOCKER + CODE INTERFACE

전제 조건 확인

```
# Docker 설치 확인
docker --version

# Docker Desktop 4.x 이상
```

대상 파일

- Redis CLI, Docker
- dmz/packages/event-engine/src/dmz/ConsumerGroupWorker.ts

핵심 인터페이스 (1분 훑기)

```
// ConsumerGroupWorker.ts 핵심 인터페이스

export interface RedisConsumerClient {
  xreadgroup(group, consumer, streams, count, blockMs): Promise<...>;
  xack(key, group, ...ids): Promise<number>;
  xautoclaim(key, group, consumer, minIdleMs, startId, count): Promise<...>;
}

// 처리 루프:
// while (running) {
//   1. reclaimPending() ← XAUTOCLAIM: idle > minIdleMs 재수신
//   2. processNew()      ← XREADGROUP ">": 새 메시지 수신
// }
```

ConsumerGroupWorker 루프 — 실행 흐름

TWO-STEP LOOP: ① RECLAIM IDLE → ② READ NEW → DISPATCH → ACK / DLQ



순서가 중요 — ① Pending 먼저 처리(장애 복구)한 뒤 ② 새 메시지 수신. 역순으로 하면 Pending이 영구 지연될 수 있다.

Step 1 — Redis 컨테이너 기동

DOCKER RUN · 5 MIN · PORT 6380 → 6379

```
# Redis 7 컨테이너 기동 (포트 6380 외부, 6379 내부)
docker run -d `
  --name kyobo-redis `
  -p 6380:6379 `
  redis:7-alpine `
  redis-server --requirepass redis_local_pw

# redis-cli 접속
docker exec -it kyobo-redis redis-cli -a redis_local_pw
```

```
127.0.0.1:6379> PING
PONG
```

이미 **kyobo-redis** 컨테이너가 있으면? `docker start kyobo-redis` 로 재기동

실습 완료 기준

- PING → PONG 확인
- redis-cli 프롬프트가 열려있음

Step 2 — XADD로 NFT 이벤트 5개 적재

XADD · 10 MIN · NFT_ISSUED × 3 + NFT_BURNED × 1 + TRANSFER × 1

```
XADD eventStream * eventType NFT_ISSUED tokenId 1 txHash 0xaaa111 recipient 0xuser1
XADD eventStream * eventType NFT_ISSUED tokenId 2 txHash 0xaaa222 recipient 0xuser2
XADD eventStream * eventType NFT_BURNED tokenId 1 txHash 0xbbb111 recipient 0xuser1
XADD eventStream * eventType NFT_ISSUED tokenId 3 txHash 0xaaa333 recipient 0xuser3
XADD eventStream * eventType TRANSFER tokenId 2 txHash 0xccc111 recipient 0xuser3
```

적재 확인

```
XLEN eventStream
```

(integer) 5

```
XRANGE eventStream - +
```

5개 이벤트 전체 내용 출력

반환값 구조 — Entry ID = "1748000001234-0" (Unix타임스탬프ms-seq번호). 이 ID가 XACK, XPENDING에서 메시지 식별자로 사용됨.

Step 3A — Consumer Group 생성

XGROUP CREATE · XINFO GROUPS · GROUP nftConsumers

```
XGROUP CREATE eventStream nftConsumers 0 MKSTREAM
```

```
# OK
```

```
# 이미 존재하면: BUSYGROUP → 정상 (RedisStreamPublisher.initialize()가 무시하는 이유)
```

```
XINFO GROUPS eventStream
```

```
# lag: 5 ← 미처리 메시지 수
```

BUSYGROUP 에러는 정상이다 — XGROUP CREATE를 두 번 실행하면 발생. RedisStreamPublisher.initialize()에서 이 오류를 catch하고 무시하기 때문에 애플리케이션이 정상 동작한다.

lag 값의 의미 — XINFO GROUPS의 lag 필드 = 이 그룹이 아직 읽지 않은(미처리) 메시지 수. 5개를 적재했으므로 lag: 5.

Step 3B — XREADGROUP으로 메시지 읽기

XREADGROUP GROUP · ">" SYMBOL · PENDING ENTRY LIST

첫 번째 읽기 - 2개

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 2 STREAMS eventStream >
```

→ 2개 반환 → PEL에 2개 등록됨

PEL 확인 (2개 등록됨)

```
XPENDING eventStream nftConsumers - + 10
```

두 번째 읽기 - 나머지 3개

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 3 STREAMS eventStream >
```

→ 3개 반환 → PEL에 총 5개 등록됨

PEL 재확인 (총 5개)

```
XPENDING eventStream nftConsumers - + 10
```

다시 XREADGROUP을 치면? → (empty array)

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 10 STREAMS eventStream >
```

">" 는 "아직 아무도 읽지 않은 새 메시지만"

5개를 모두 읽었으므로, 새 메시지가 없음

XREADGROUP — 실행 흐름

COMMAND → REDIS INTERNAL → PEL REGISTRATION → CONSUMER RECEIPT

① 명령 입력

GROUP nftConsumers

Consumer Group 지정

consumer-1

수신 Consumer 이름

">" 심볼

아무도 안 읽은 메시지만

COUNT 10

최대 10개 배치

② Redis 내부 처리

미전달 메시지 탐색

스트림에서 읽히지 않은 항목

PEL 등록

consumer-1 소유 기록

delivery count +1

있음 → 메시지 반환

없음 → [] 반환

③ Consumer 처리

비즈니스 로직 실행

processor.process(msg)

XACK → PEL 제거

처리 완료 확인

XACK 없이 crash →

PEL 잔류 → XAUTOCLAIM 대상

XACK 전까지 메시지는 PEL에 잔류 — crash 후 재시작해도 XREADGROUP "0"으로 재수신 가능. At-least-once 보장의 핵심 메커니즘.

">" vs "0" — 이것이 핵심이다

NEW MESSAGES vs PEL REDELIVERY — TWO DIFFERENT MODES

">" — 새 미처리 메시지만

- 아직 어떤 Consumer도 읽지 않은 메시지
- 처음 호출 → 3개 반환
- 두 번째 호출 → 0개 반환 (이미 읽었으므로)
- 정상 운영 루프에서 사용

"0" (또는 "0-0") — PEL 재읽기

- 해당 Consumer가 읽었지만 XACK 안 한 것들
- consumer-1의 PEL에 있는 메시지 재반환
- 3개 다시 반환됨
- crash 후 재시작 복구에 사용

"0" 사용 예시 - crash 복구 시나리오

```
XREADGROUP GROUP nftConsumers consumer-1 COUNT 10 STREAMS eventStream 0
```

→ consumer-1의 PEL에 있는 메시지 재반환 (XACK 안 한 것들)

→ 5개 다시 반환됨

실습 완료 기준 — XREADGROUP > → 2개, 3개 순서로 수신 | 두 번째 > → empty | XINFO GROUPS에서 pending 5 확인

Step 4A — XPENDING으로 PEL 현황 조회

PENDING ENTRY LIST — OWNER · IDLE TIME · DELIVERY COUNT

```
XPENDING eventStream nftConsumers
```

```
# 1) (integer) 5          ← 미처리 메시지 총 수
# 2) "1748000001234-0"    ← 가장 오래된 미처리 ID
# 3) "1748000005678-0"    ← 가장 최근 미처리 ID
# 4) 1) 1) "consumer-1"
#      2) "5"              ← consumer-1이 소유한 미처리 수
```

```
# 상세 조회 (처음부터 끝까지, 최대 10개)
```

```
XPENDING eventStream nftConsumers - + 10
```

```
# 1) 1) "1748000001234-0"    ← messageId
#      2) "consumer-1"       ← 소유 Consumer
#      3) (integer) 1500      ← idle time (ms) ← XREADGROUP 이후 경과 시간
#      4) (integer) 1         ← delivery count (배달 횟수)
# 2) 1) "1748000002345-0" ...
# ... (총 5개)
```

Step 4B — XACK 후 PEL 변화 확인

XACK REMOVES FROM PEL · STREAM ITSELF IS PRESERVED

```
# 첫 번째 메시지 ACK (실제 반환된 ID로 교체)
XACK eventStream nftConsumers <Entry ID 1>
# (integer) 1 ← ACK된 수

# PEL 재확인
XPENDING eventStream nftConsumers - + 10
# 4개로 줄어듦 (첫 번째 제거됨)

# 2개 동시 ACK
XACK eventStream nftConsumers <Entry ID 2> <Entry ID 3>
# (integer) 2

# 나머지 전부 ACK
XACK eventStream nftConsumers <Entry ID 4> <Entry ID 5>

XPENDING eventStream nftConsumers - + 10
# (empty array) ← PEL 비어있음 ✓
```

XACK 전

PEL: [Entry ID 1,
Entry ID 2,
... (총 5개)]



XACK 후

PEL: [Entry ID 2,
... (4개 남음)]
← 첫 번째 제거됨

중요: XACK는 PEL에서만 제거. 스트림 자체는 보존 → XRANGE로 확인 가능.

Step 5A — 미ACK 재수신 시나리오

CONSUMER CRASH SIMULATION · XAUTOCLAIM OWNERSHIP TRANSFER

현재 상황: PEL: [1714000002000-0 (consumer-1), 1714000003000-0 (consumer-1)]

consumer-1 (crash)

- 두 메시지를 읽었으나 처리 중 crash
- XACK를 보내지 못함
- minIdleMs(30,000ms) 경과

XAUTOCLAIM



consumer-2 (새로 기동)

- XAUTOCLAIM으로 두 메시지 인계받음
- 소유자 변경: consumer-1 → consumer-2
- delivery count 증가 (1→2)

실습에서는 idleMs를 1ms로 설정해 즉시 인계받는 것을 시뮬레이션. 실제 운영에서는 30,000ms(30초) 이상 설정.

Step 5B — XAUTOCLAIM 실행 및 결과 확인

XAUTOCLAIM · OWNERSHIP CHANGE · DELIVERY COUNT INCREMENT

```
XAUTOCLAIM eventStream nftConsumers consumer-2 1 0-0
```

```
# 결과 (예시):
```

```
# 1) "0-0" ← nextId (더 이상 없으면 0-0)
```

```
# 2) 1) 1) "1748000002345-0" ← 인계받은 메시지들
```

```
# 2) 1) "eventType"
```

```
# 2) "NFT_ISSUED"
```

```
# ...
```

```
# 2) ... (PEL에 있던 메시지 모두 인계)
```

```
# 3) (empty array) ← 삭제된 메시지 없음
```

```
# PEL 재확인
```

```
XPENDING eventStream nftConsumers - + 10
```

```
# 1) ...
```

```
# 2) "consumer-2" ← 소유자: consumer-1 → consumer-2 변경됨
```

```
# 3) (integer) 0 ← idle time 리셋
```

```
# 4) (integer) 2 ← delivery count 증가 (1→2)
```

```
# ...
```

delivery count vs retryCount — 두 가지 카운터

REDIS PEL DELIVERY COUNT vs MESSAGE FIELD RETRY COUNT

delivery count (Redis 관리)

- XPENDING에서 확인 가능
- XREADGROUP/XAUTOCLAIM 호출마다 자동 증가
- Redis가 자동으로 관리
- ConsumerGroupWorker가 직접 읽지 않음

retryCount (메시지 필드)

- 메시지 필드 `_retryCount`에 기록
- ConsumerGroupWorker가 직접 증가
- 3회 초과 시 DLQ로 이동
- XPENDING에는 보이지 않음

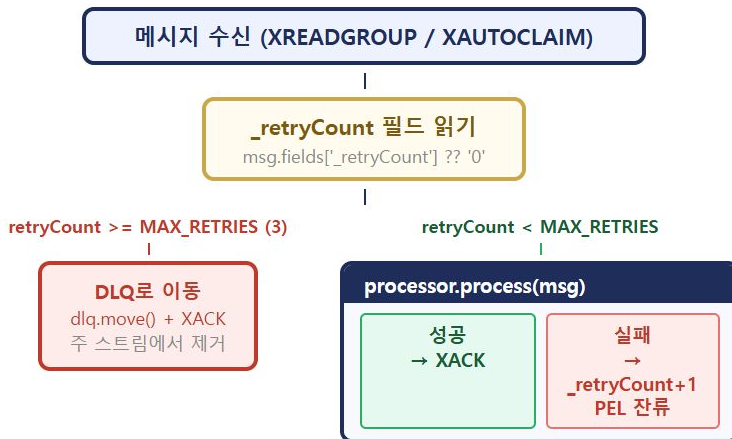
```
// ConsumerGroupWorker._handleWithRetry()
const retryCount = parseInt(msg.fields['_retryCount'] ?? '0', 10);

if (retryCount >= this.MAX_RETRIES) {
  // 3회 초과 → DLQ로 이동
  await this.dlq.move({ ... });
  await this.redis.xack(streamKey, groupName, msg.id);
  return;
}
```

두 가지를 같이 봐야 전체 그림이 완성됨 — delivery count는 Redis가 관리, retryCount는 메시지 필드에 직접 기록.

_handleWithRetry() — 실행 흐름

RETRY COUNT CHECK → PROCESS OR DLQ → XACK ON SUCCESS



두 카운터의 역할 분리 — `delivery count`(Redis 자동 관리)는 XPENDING에서 확인, `retryCount`(메시지 필드)는 비즈니스 재시도 한도 관리. 둘을 같이 봐야 전체 그림 완성.

Step 5C — 인계받은 메시지 XACK + PEL 클리어

FINAL XACK · PEL EMPTY · STEP 5 COMPLETION

```
# consumer-2가 처리 완료 (실제 반환된 ID들로 교체)
XACK eventStream nftConsumers <E1> <E2> <E3> ...

XPENDING eventStream nftConsumers
# 1) (integer) 0 ← PEL이 비어있음 ✓
```

Step 5 완료 기준

- XAUTOCLAIM으로 consumer-2가 2개 메시지 인계받음 확인
- XPENDING에서 소유자가 consumer-2로 변경됨 확인
- delivery count가 2로 증가됨 확인
- 최종 XACK 후 PEL이 0개됨 확인

이것이 **At-least-once** 보장의 핵심 — XACK 없이 crash한 Consumer의 메시지가 XAUTOCLAIM으로 다른 Consumer에게 재전달됨. 메시지 유실 없음.

Step 6A — 이벤트 6개 추가 + 터미널 2개 준비

HORIZONTAL SCALING · TWO TERMINALS · 10 MIN

```
# 스트림 초기화
```

```
DEL eventStream
```

```
XGROUP CREATE eventStream nftConsumers 0 MKSTREAM
```

```
XADD eventStream * eventType NFT_ISSUED tokenId 1 txHash 0xeeee001
```

```
XADD eventStream * eventType NFT_ISSUED tokenId 2 txHash 0xeeee002
```

```
XADD eventStream * eventType NFT_ISSUED tokenId 3 txHash 0xeeee003
```

```
XADD eventStream * eventType NFT_BURNED tokenId 1 txHash 0xffff001
```

```
XADD eventStream * eventType TRANSFER tokenId 2 txHash 0xffff002
```

```
XADD eventStream * eventType NFT_ISSUED tokenId 4 txHash 0xeeee004
```

```
XLEN eventStream
```

```
# (integer) 6
```

```
# 터미널 2 - 새 PowerShell 창 열고 접속 (폴더 무관)
```

```
docker exec -it kyobo-redis redis-cli -a redis_local_pw
```


Step 6B — 두 Consumer 동시 읽기

CONSUMER GROUP AUTO-DISTRIBUTION · FIRST-COME FIRST-SERVED

터미널 1 (consumer-1)

```
XREADGROUP GROUP nftConsumers
consumer-1 COUNT 3
STREAMS eventStream >
# 결과: Entry 1~3 (NFT_ISSUED tokenId=1,2,3)
```

터미널 2 (consumer-2)

```
XREADGROUP GROUP nftConsumers
consumer-2 COUNT 10
STREAMS eventStream >
# 결과: Entry 4~6 (NFT_BURNED/TRANSFER/NFT_ISSUED)
```

6개가 3개씩 분배됨 — 같은 Consumer Group 안에서 각 Consumer가 서로 다른 메시지를 가져감. 중복 수신 없음.

라운드 로빈이 아님 — 먼저 XREADGROUP을 호출하는 Consumer가 먼저 가져간다. 두 Consumer가 동시에 호출하면 Redis가 내부적으로 순서를 결정한다.

Step 6C — 분배 확인 + Consumer 3 시뮬레이션

XPENDING OWNER VERIFICATION · HORIZONTAL SCALE-OUT

```
XPENDING eventStream nftConsumers - + 10
```

```
# 1) Entry ID → "consumer-1" (NFT_ISSUED tokenId=1)
# 2) Entry ID → "consumer-1" (NFT_ISSUED tokenId=2)
# 3) Entry ID → "consumer-1" (NFT_ISSUED tokenId=3)
# 4) Entry ID → "consumer-2" (NFT_BURNED tokenId=1)
# 5) Entry ID → "consumer-2" (TRANSFER tokenId=2)
# 6) Entry ID → "consumer-2" (NFT_ISSUED tokenId=4)
```

```
# Consumer 3 추가 (선택적, 새 PowerShell 창)
```

```
docker exec -it kyobo-redis redis-cli -a redis_local_pw
```

```
XREADGROUP GROUP nftConsumers consumer-3 COUNT 4 STREAMS eventStream >
```

```
# → 새 메시지가 있다면 consumer-3이 가져감
```

```
# → 수평 확장: Consumer 수만큼 처리량 증가 (I/O bound 처리 기준)
```

Step 6 완료 기준

- 두 Consumer가 6개 메시지를 3개씩 나눠 가져감 확인
- XPENDING에서 소유자 구분 확인 (consumer-1 / consumer-2)
- 같은 메시지를 두 Consumer가 중복 수신하지 않음 확인
- Consumer Group이 자동 분배 (먼저 읽는 쪽) 확인

At-least-once vs Exactly-once

DELIVERY GUARANTEE · CLI VERIFICATION POINT

At-least-once (Redis Streams 기본)

- XACK 없이 재시작하면 PEL에서 재수신
- 중복 처리 가능성 있음
- 메시지 유실 없음 (강한 내구성)
- XAUTOCLAIM으로 crash 복구

Exactly-once (애플리케이션 책임)

- Redis Streams 자체로는 보장 불가
- 멱등성(Idempotency) 처리 필요
- requestId로 중복 차단
- → S9 설계 원칙 참조

CLI 실습 포인트 — XACK 없이 재시작하면 PEL에서 메시지가 재수신되는 것을 직접 확인한다.

S9 참조 — At-least-once 설계 원칙, XACK 순서 불변 규칙(처리 → 커밋 → XACK), Exactly-once가 불가능한 이유를 S9에서 상세히 다룬다.

DLQ 패턴 + 수평 확장 메커니즘

DEAD LETTER QUEUE · HORIZONTAL SCALING · CLI FOCUS

DLQ (Dead Letter Queue) 패턴

- XPENDING에서 delivery count가 3 이상인 메시지
- → DLQ로 이동 후 원 스트림에서 XACK
- _handleWithRetry() 코드 구현 → S9 참조
- DLQHandler.ts (move / listPending / requeueMessage) → S11 참조

CLI 포인트: XPENDING에서 delivery count 3 이상인 메시지가 DLQ로 이동되는 흐름을 S11 실습에서 직접 확인.

수평 확장 메커니즘

- Consumer 수 증가 → 처리량 비례 증가
- I/O bound 처리 기준
- 같은 Consumer Group에 Consumer 추가만으로 확장
- 코드 변경 없음, 설정 변경 없음

CLI 포인트: 터미널 2개에서 동시에 XREADGROUP → 메시지 자동 분배 확인. 멍등성 안전성 → S9 참조.

강의 강조 포인트 3가지

THREE KEY CONCEPTS TO EMPHASIZE IN CLI PRACTICE

">" vs "0-0" — 목적에 따라 다르다

XREADGROUP의 >는 새 미처리 메시지만. XAUTOCLAIM의 0-0은 처음부터 전체 PEL 스캔. 혼동하면 메시지를 놓치거나 중복 처리한다.

delivery count — XPENDING에서 증가하는 수치

XPENDING에서 확인하는 delivery count는 Redis가 관리. _retryCount(메시지 필드)는 ConsumerGroupWorker가 직접 기록. 두 개는 다른 개념이다.

PEL = 처리 중 영수증

XACK 전까지 Consumer 소유. 재시작 후 XAUTOCLAIM으로 복구. PEL이 없으면 Consumer crash 시 메시지가 유실된다.

이론 → CLI 정리표 + S5~S10 파이프라인

THEORY-TO-CLI MAPPING · FULL PIPELINE SUMMARY

S7 이론	S8 CLI 명령	확인 내용
Append-only 로그	XADD ... *	messageId 자동 생성
Consumer Group	XGROUP CREATE	XINFO GROUPS로 확인
">" 심볼	XREADGROUP ... >	첫 번째: 3개, 두 번째: 0개
PEL 등록	XREADGROUP 후	XPENDING으로 목록 확인
처리 완료 선언	XACK	XPENDING에서 제거 확인
Consumer crash 복구	XAUTOCLAIM 1 0-0	소유자 변경 + delivery count 증가
수평 확장	두 터미널에서 XREADGROUP	메시지 분배 확인

핵심 명령 정리 + 다음 세션 예고

COMMAND CHEAT SHEET · NEXT SESSION PREVIEW S9~S11

명령	역할	기억할 것
XADD ... *	스트림에 항목 추가	* = ID 자동 생성
XLEN	스트림 항목 수	
XRANGE - +	전체 항목 조회	읽어도 삭제 안 됨
XGROUP CREATE ... 0	그룹 생성, 처음부터	\$ = 지금 이후만
XREADGROUP ... >	새 메시지 + PEL 등록	> 중요
XREADGROUP ... 0	내 PEL 재읽기	crash 복구용
XPENDING	PEL 상태 조회	소유자 + delivery count
XACK	처리 완료 → PEL 제거	반드시 처리 후에
XAUTOCLAIM	idle 초과 메시지 인계	minIdleMs 설정 중요
XINFO GROUPS	그룹 상태 요약	lag = 미처리 메시지 수

다음 세션 예고

- **S9** — At-least-once 설계 원리, XACK 순서 불변 규칙, Exactly-once 불가능한 이유
- **S10** — ConsumerGroupWorker 코드 상세: start() / _reclaimPending() / _processNew() / _handleWithRetry(), DLQ 연동, retryCount 증가 로직
- **S11** — DLQ 설계와 운영: DLQHandler.ts 내부 구조, move / listPending / requeueMessage

예상 Q&A

ANTICIPATED QUESTIONS — THREE COMMON ONES

Q1. XRANGE로 조회하면 메시지가 삭제되는가?

삭제되지 않는다. Redis Streams는 append-only 로그다. XRANGE는 읽기 전용 조회다. XACK도 PEL에서만 제거하고 스트림 자체는 보존한다. 스트림에서 실제로 삭제하려면 XDEL을 직접 호출하거나 MAXLEN으로 제한해야 한다.

Q2. Consumer Group을 삭제하면 PEL도 사라지는가?

사라진다. XGROUP DESTROY eventStream nftConsumers를 실행하면 그룹과 함께 PEL 전체가 삭제된다. 스트림 자체는 남는다. 그룹을 다시 만들면 PEL은 비어있는 상태로 시작한다.

Q3. XAUTOCLAIM의 startId를 0-0이 아닌 특정 ID로 설정하면?

해당 ID 이후의 메시지부터만 검색한다. 0-0은 스트림의 처음을 의미. 대량 PEL에서는 nextId로 페이지네이션 처리 가능:

```
XAUTOCLAIM ... 30000 0-0 COUNT 10 → nextId 반환 → XAUTOCLAIM ... 30000 <nextId> COUNT 10
```